Lógica de Programação

Prof. Relton Alves da Silva
E-mail: relton.silva@ulife.com.br

Algoritmos

Um algoritmo não é a solução do problema, pois, se assim fosse, cada problema teria um único algoritmo.

Podemos definir um algoritmo como um caminho para a solução de um determinado problema.

Exemplos de algoritmos:

instruções de montagem, receitas, manuais de uso, etc.

Algoritmos

Um algoritmo é uma sequência de instruções finita e ordenada de forma lógica para a resolução de uma determinada tarefa ou problema.

Algoritmos não se aprendem:

- Copiando algoritmos
- Estudando algoritmos

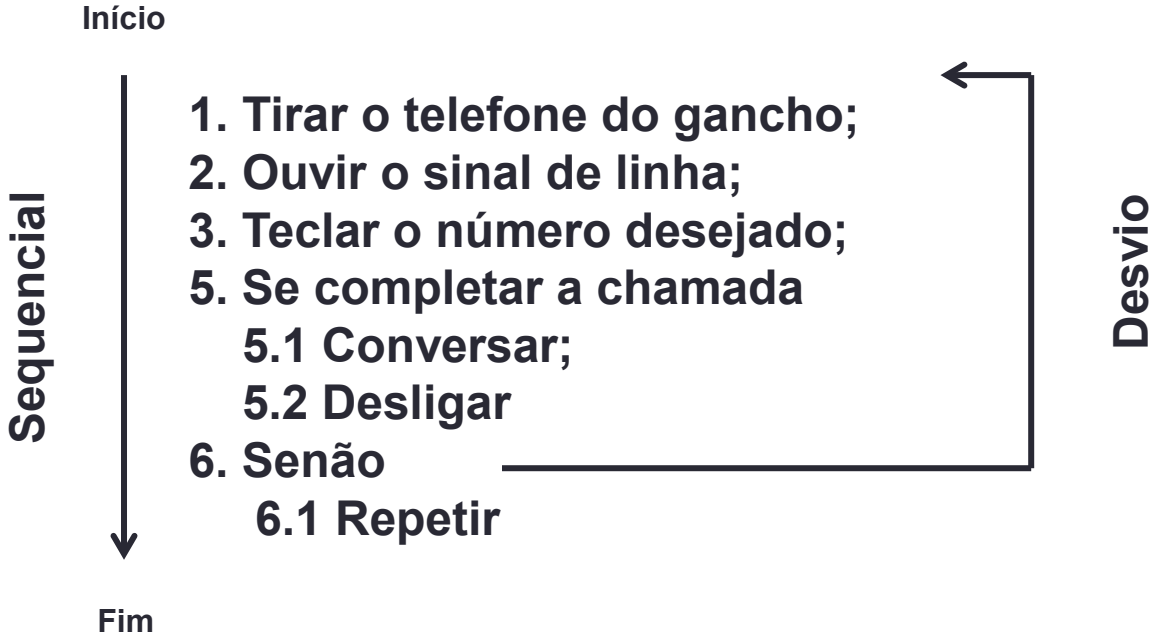
Algoritmos só se aprendem:

- Construindo algoritmos
- Testando algoritmos

Algoritmos não computacionais

Um algoritmo não computacional é um algoritmo cuja sequência de passos, a princípio, não pode ser executada por um computador.

Exemplo:



Algoritmos não computacionais

Um outro exemplo típico de algoritmo é uma receita culinária, como no exemplo abaixo:

Algoritmo para fritar um ovo (versão 1.0)

1. Colocar um ovo na frigideira
2. Esperar o ovo ficar frito
3. Remover o ovo da frigideira

Algoritmos não computacionais

Para evitar “ambiguidade”, o algoritmo anterior poderia ser mais detalhado e completo.

Versão 2.0 do “Algoritmo para fritar um ovo”

1. Retirar um ovo da geladeira
2. Colocar a frigideira no fogo
3. Colocar óleo na frigideira
4. Esperar até o óleo ficar quente
5. Quebrar o ovo separando a casca
6. Colocar o conteúdo do ovo na frigideira
7. Esperar um minuto
8. Retirar o ovo da frigideira
9. Apagar o fogo

Algoritmos

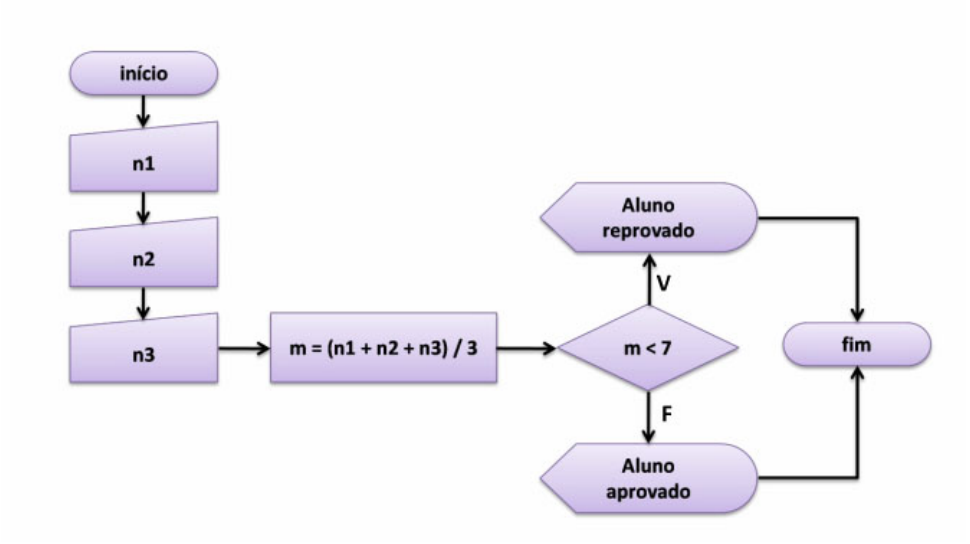
Ao longo dos anos surgiram muitas formas de representar os algoritmos, algumas utilizando **linguagens semelhantes às linguagens de programação**.

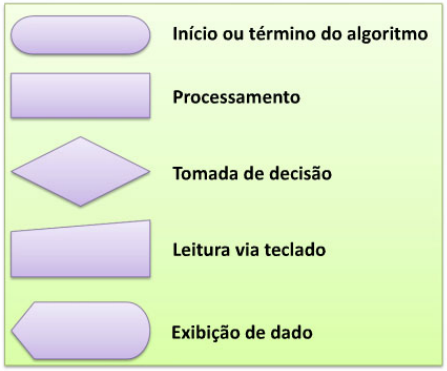
```
leia n1
leia n2
calcule m = (n1 + n2)/2
se m < 7 então
  escreva "Aluno reprovado"
senão
  escreva "Aluno aprovado"
```

Pseudocódigo

Algoritmos

Formas gráficas também são frequentemente utilizadas para representar algoritmos.





Fluxograma

Variável

Uma variável pode ser definida como tudo aquilo que está sujeito a variações, que é incerto, instável ou inconstante. Ou seja, uma variável trata-se de um conteúdo que não é fixo ou permanente.

Observação: Embora o “valor” de uma variável possa ser alterado diversas vezes, o conteúdo deve ser sempre do mesmo tipo*.

Exemplos:

compra, venda: real

nome: caractere

* No *Portugol,* os tipos de **dados primitivos** são: inteiro, real, caractere e lógico.

Portugol - O que é?

Para que o algoritmo possa ser executado por um computador é importante que as instruções sejam corretas e sem ambiguidades.

- O **português estruturado** (*Portugol*) é uma simplificação extrema do Português, limitada a umas poucas palavras e estruturas que têm um significado muito bem definido.
- A ***sintaxe do portugol*** consiste no conjunto de palavras e regras que definem o formato das sentenças válidas.

Palavras Reservadas - O que são?

Palavra reservada é toda e qualquer palavra que foi feita especialmente para o compilador de uma linguagem específica.

O programador não poderá utilizar uma palavra reservada para outro fim que não o definido pelo compilador*

* A não ser que seja ela utilizada como caractere (atribuída como valor de uma variável, por exemplo).

Lista de Palavras Reservadas

PALAVRAS RESERVADAS			
aleatorio	e	grauprad	passo
abs	eco	inicio	pausa
algoritmo	enquanto	int	pi
arccos	entao	interrompa	pos
arcsen	escolha	leia	procedimento
arctan	escreva	literal	quad
arquivo	exp	log	radpgrau
asc	faca	logico	raizq
ate	falso	logn	rand
caracter	fimalgoritmo	maiusc	randi
caso	fimenquanto	mensagem	repita
compr	fimescolha	minusc	se
copia	fimfuncao	nao	sen
cos	fimpara	numerico	senao
cotan	fimprocedimento	numpcarac	timer
cronometro	fimrepita	ou	tan
debug	fimse	outrocaso	verdadeiro
declare	função	para	xou

Algoritmos Computacionais

A princípio, precisamos ter em mente que o computador não executa nada!

Para que ele faça uma determinada tarefa, como calcular uma folha de pagamento, por exemplo, é necessário que ele execute um **programa**.

Um programa nada mais é do que um algoritmo computacional descrito em uma linguagem de programação.

Linearização de Expressões

Para a construção de algoritmos que realizam cálculo matemáticos, todas as expressões aritméticas devem ser linearizadas (colocadas em linha).

Exemplo $\left\{\left[\frac{2}{3}-(5-3)\right]+1\right\}.5$	`((2/3-(5-3))+1)*5`
Tradicional	**Computacional**

Operadores Aritméticos

As tabelas seguintes mostram os operadores aritméticos disponíveis no Português Estruturado.

OPERADORES ARITMÉTICOS	PORTUGUÊS ESTRUTURADO
Adição	**+**
Subtração	**-**
Multiplicação	*****
Divisão	**/**
Divisão Inteira	****
Exponenciação	**^ ou Exp** (*<base>,<expoente>*)
Módulo (resto da divisão)	**%**

Exemplo

Crie um algoritmo que:

a) Solicite que o usuário informe o seu nome ;

b) Guarde o nome do usuário na respectiva variável;

c) Exiba a mensagem: seja bem vindo ‘nome-usuário’

Sintaxe do exemplo

```
algoritmo "Boas Vindas"
// Função :  Exibir mensagem de boas vindas ao usuário
// Autor :   Prof. Rogério
var
nome: caracter
inicio
// Seção de Comandos
escreval ("Digite o seu nome: ")
leia (nome)
escreva ("Seja bem vindo ", nome)
fimalgoritmo
```

Operadores Relacionais

Os operadores relacionais realizam a comparação entre dois operandos ou duas expressões e resultam em valores lógicos (**VERDADEIRO** ou **FALSO**).

OPERADORES RELACIONAIS	PORTUGUÊS ESTRUTURADO
Maior	>
Menor	<
Maior ou igual	>=
Menor ou igual	<=
Igual	=
Diferente	<>

Exemplo
2+5>4 resulta **VERDADEIRO**
3<>3 resulta **FALSO**

Operadores de Atribuição

Para “colocar” um valor em uma variável dentro de um algoritmo, utilizamos os operadores de atribuição. Os operadores de atribuição são representados por uma seta (<-) apontando para a esquerda ou por dois ponto sucedidos pelo sinal de igual (:=).

Exemplos

Peso <- 70 // Este comando atribui à variável Peso o valor 70

Media <- (Nota1+ Nota2) / 2 // O resultado do cálculo é atribuído à média

Exercício - Controle de Estoque

Crie um algoritmo que:

a) Solicite que a quantidade de produtos comprados seja informada;

b) Solicite que a quantidade de produtos vendidos seja informada;

c) Realize a subtração entre os valores comprados e vendidos e, em seguida, exiba o resultado (quantidade em estoque)

Sintaxe do exercício (estoque)

```
algoritmo "Controle de Estoque"
// Função :  Calcular o estoque (comprados - vendidos)
// Autor :   Prof. Rogério
var
compra, venda, estoque: inteiro
inicio
escreval ("Digite a quantidade de produtos comprados: ")
leia (compra)
escreval ("Digite a quantidade de produtos vendidos: ")
leia (venda)
estoque <- (compra - venda)
escreva ("A quantidade em estoque é ", estoque)
fimalgoritmo
```

Revendo alguns Conceitos

- A instrução ***escreva*** solicita ao usuário para que insira, no programa, alguma informação. Por exemplo, “Digite o seu nome”
- Quando o usuário insere a informação, o algoritmo a recebe e guarda na variável através da instrução ***leia***
- Para exibir o resultado do processamento utilizamos novamente a instrução ***escreva***

Comandos de E/S (Entrada/Saída)

Em *Portugol*, usamos o comando **escreva** para à exibição de valores na tela (solicitações ou resultado do processamento).

OBS.: No Visualg existem dois comandos escreva com finalidades diferentes quando usado consecutivamente.

Escreva () // Mostra o resultado na mesma linha, mas em colunas diferentes.

Escreval () // Mostra o primeiro resultado na mesma linha depois em linhas diferentes.

Tipo de dado: Caractere

Os dados literais são formados por um único **caractere** ou uma sequência de caracteres, que podem ser letras (maiúsculas ou minúsculas), números ou símbolos especiais (como #, $, @, ?, &, entre outros).

Os números, quando representados como caracteres, **não podem ser utilizados para cálculos**.

Dados numéricos

No estudo da matemática, você aprendeu que existem diversos conjuntos numéricos (conjunto dos números naturais, inteiros, racionais, irracionais e reais).

No estudo dos algoritmos, lidaremos apenas com os números **inteiros** e os números **reais**.

Tipo de dado: Inteiro

Os números inteiros são os números positivos ou negativos que não possuem parte decimal ou fracionária. Ex.: 15, -487, 0, 27835, -14937, 100.

Em algoritmos, os números inteiros são utilizados geralmente para expressar valores, como quantidades ou idade.

Tipo de dado: Real

Os números reais são os números positivos ou negativos que englobam números decimais ou fracionários.

Ex.: 15, -487, 1.78, 0.254, 27835, 100, 8.50.

Nas linguagens de programação são chamados de pontos flutuantes. Valores reais são aplicáveis em algoritmos que manipulam dados que expressam valores fracionários, como salário, média, preço, porcentagem, entre outros.

Tipo de dado: Lógico

Os dados lógicos são também chamados de booleanos, por ter sua origem na álgebra booleana. Os valores que esses tipos de dados podem assumir são **Verdadeiro** ou **Falso**, podendo representar apenas um desses dois valores.

Um exemplo simples:

```
algoritmo "logicos"
var
igualdade: logico
inicio
igualdade <- 10 = 12
escreva (igualdade)
fimalgoritmo
```

Estrutura condicional

Até agora nossos códigos executavam todas as linhas do começo ao final. Porém, com o comando SE podemos bifurcar o código, ou seja, escolher um caminho em tempo de execução.

Exemplo: se a nota for menor que 7, aluno reprovado, se não, aluno aprovado.

A sintaxe do comando SE é a seguinte:

se <expressão booleana> **entao**
  // Códigos para o caso verdadeiro
senao
  // Códigos para o caso falso
fimse

Exercício - Média Escolar

Crie um algoritmo que:

a) Solicite que o usuário digite a nota do 1º Bimestre;

b) Solicite que o usuário digite a nota do 2º Bimestre;

c) Calcule a média e;

d) Exiba o valor da média;

e) Se a média for maior ou igual a 7 exiba a mensagem "Aluno Aprovado";

f) Se a média for menor que 7 exiba a mensagem "Aluno Reprovado".

Sintaxe do exercício (média escolar)

```
algoritmo "Calcular Média"
// Função :  Calcular a média a partir de duas notas
// Autor :   Prof. Rogério
var
nota1, nota2, media: real
inicio
escreval ("Digite a nota do 1º bimestre: ")
leia (nota1)
escreval ("Digite a nota do 2º bimestre: ")
leia (nota2)
media <- (nota1 + nota2) / 2
escreva ("A média é ", media)
se media >= 7 entao
escreva (" Aluno Aprovado")
senao
escreva (" Aluno Reprovado")
fimse
fimalgoritmo
```

Operadores Lógicos

Os operadores lógicos atuam sobre expressões e também resultam em valores lógicos (**VERDADEIRO** ou **FALSO**).

OPERADORES LÓGICOS	PORTUGUÊS ESTRUTURADO	SIGNIFICADO
Multiplicação lógica	E	Resulta VERDADEIRO se ambas as partes forem verdadeiras.
Adição lógica	Ou	Resulta VERDADEIRO se uma das partes é verdadeira.
Negação	Nao	Nega uma afirmação, invertendo o seu valor lógico: se for VERDADEIRO torna-se FALSO, se for FALSO torna-se VERDADEIRO.

tabela-verdade

A tabela exibe os resultados das aplicações dos operadores lógicos, conforme os valores dos operadores envolvidos.

A	B	A E B	A OU B	NÃO A	NÃO B
VERDADEIRO	VERDADEIRO	VERDADEIRO	VERDADEIRO	FALSO	FALSO
VERDADEIRO	FALSO	FALSO	VERDADEIRO	FALSO	VERDADEIRO
FALSO	VERDADEIRO	FALSO	VERDADEIRO	VERDADEIRO	FALSO
FALSO	FALSO	FALSO	FALSO	VERDADEIRO	VERDADEIRO

Exemplo
(2+5>4) e (3<>3) resulta **FALSO**, pois **VERDADEIRO** e **FALSO** resulta **FALSO**.

Operador de módulo

O operador de módulo é utilizado para identificar o resto de uma divisão inteira, por exemplo:

Identificar se o número digitado pelo usuário é positivo ou negativo

se (num) MOD 2 = 0 **entao**

  escreval ("Este número é par")

senao

  escreval ("Este número é impar")

Estruturas de repetição

O VisuAlg implementa as três estruturas de repetição mais usadas nas linguagens de programação:

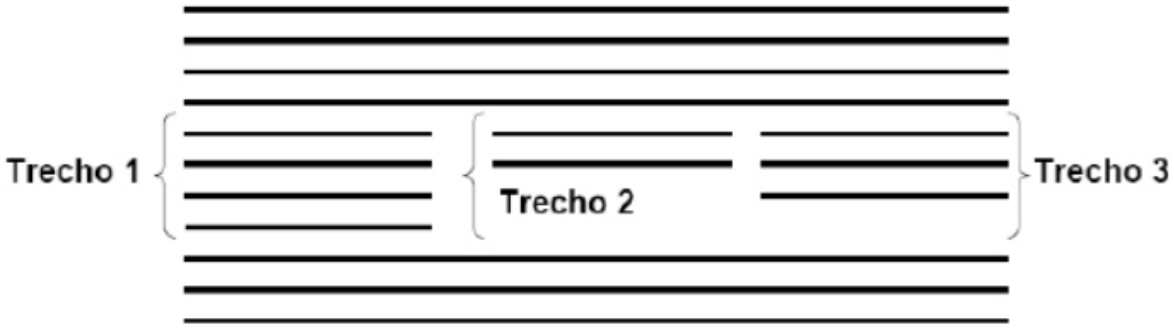
1. **laço:** para...ate...faca
2. **laço condicional:** enquanto...faca
3. **repita:** ate

Exemplo: Criar um algoritmo que conte de 1 até 10 e exibe os números em ordem crescente (incremental)

```
algoritmo “Contar de 1 até 10"
var contador: inteiro
inicio
para contador de 1 ate 10 faca
escreva (contador)
fimpara
fimalgoritmo
```

Comando de Seleção Múltipla

Em algumas situações, ao chegar em uma determinada Instrução, o algoritmo deve selecionar um dentre alguns trechos, tendo como base para esta escolha um conjunto de valores.



Para lidar com casos deste tipo, foi criado um comando específico para seleção múltipla

Comando de Seleção Múltipla

Sintaxe:

escolha (<variável>)
caso <valor11>, <valor12>,..., <valorn>
<sequencia de comandos1>
caso <valor21>, <valor22>,..., <valorn>
<sequencia de comandos2>
outrocaso
<sequencia de comandos extra>
fimescolha

Comando de Seleção Múltipla

Exemplo:

```
algoritmo "Times do Campeonato Brasileiro"
var time: caractere
inicio
escreva ("Entre com o nome de um time de futebol: ")
leia (time)
escolha time
caso "São Paulo", "Palmeiras", "Santos", "Corínthians“
escreval ("É um time paulista.")
caso "Flamengo", "Fluminense", "Vasco", "Botafogo"
escreval ("É um time carioca.")
outrocaso
escreval (“Time de outro estado.")
fimescolha
fimalgoritmo
```

Estruturas de repetição

No VisuAlg, se o valor inicial de uma sequência for maior que o valor final, o laço não será executado nenhuma vez.

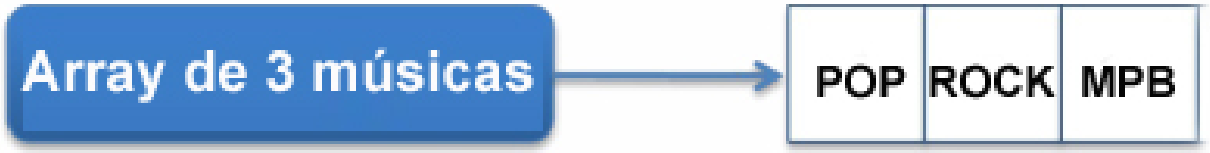
Como não existe um operador específico para decremento no *Portugol*, no exercício anterior, para que o contador conte de 10 até 1 será necessário adotar um “passo - 1”

```
algoritmo “Contar de 10 até 1"
var contador: inteiro
inicio
para contador de 10 ate 1 passo - 1 faca
escreva (contador)
fimpara
fimalgoritmo
```

Tipos de dados avançados: vetores

- Um vetor (ou array) agrupa variáveis com o mesmo tipo de dado;
- Tem um tamanho pré-determinado.

Exemplo:



Tipos de dados avançados: vetores

Para declarar uma variável como vetor precisamos fornecer as seguintes informações:

- O tipo de dado que será armazenado em cada posição;
- O tamanho do vetor.

Exemplo:

Os colchetes determinam que é um vetor

IDADE: VETOR [1..5] DE **INTEIRO**

Tipo de dado

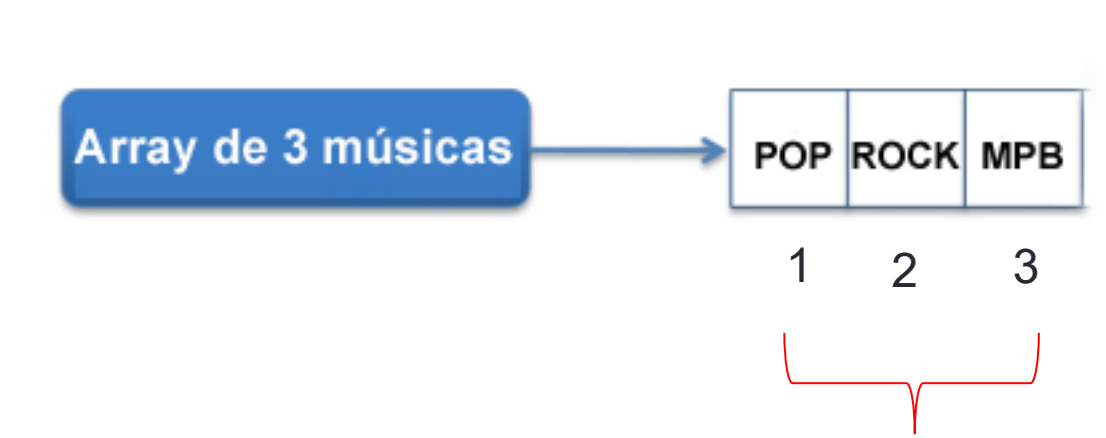
Nome do Vetor

Tamanho do vetor

Índices de um vetor

Cada elemento ocupa uma posição dentro do vetor, permitindo a sua diferenciação.

- A posição dos elementos é numerada



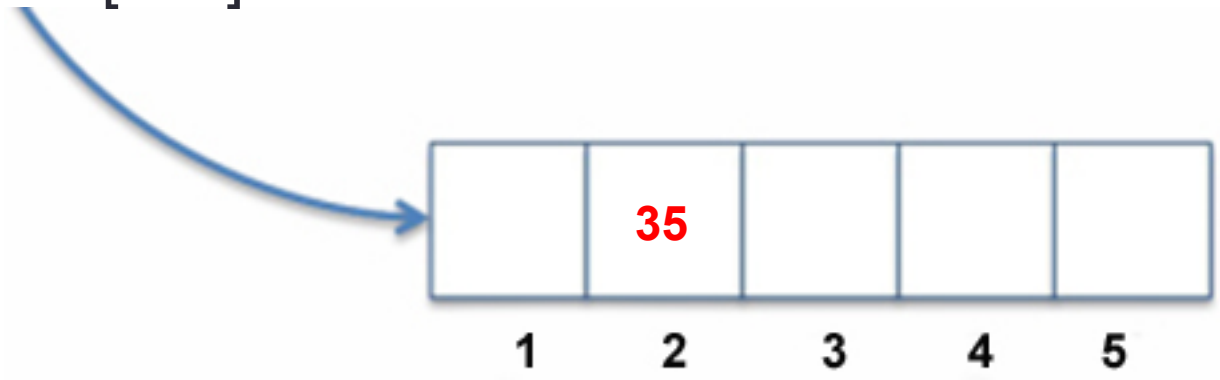
Tamanho do Vetor: três posições

Atribuindo valores a um vetor

Como um vetor é indexado, um valor deve sempre ocupar uma posição (um valor não pode ser atribuído a um vetor).

- O valor é acessado pela [posição]

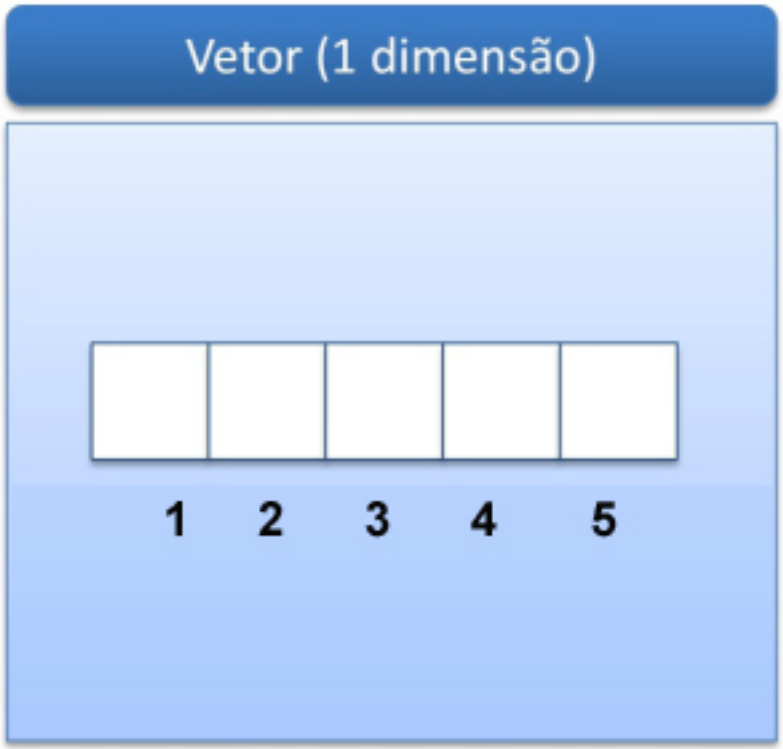
IDADE: VETOR [1..5] **DE INTEIRO**

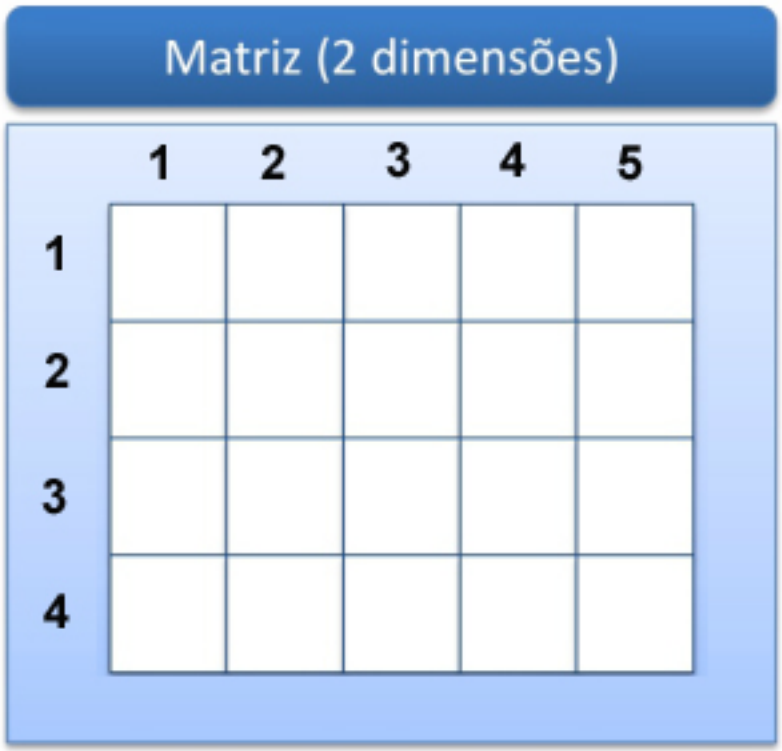


idade[2] <- 35
escreva (vetor[2])

Tipos de dados avançados: matriz

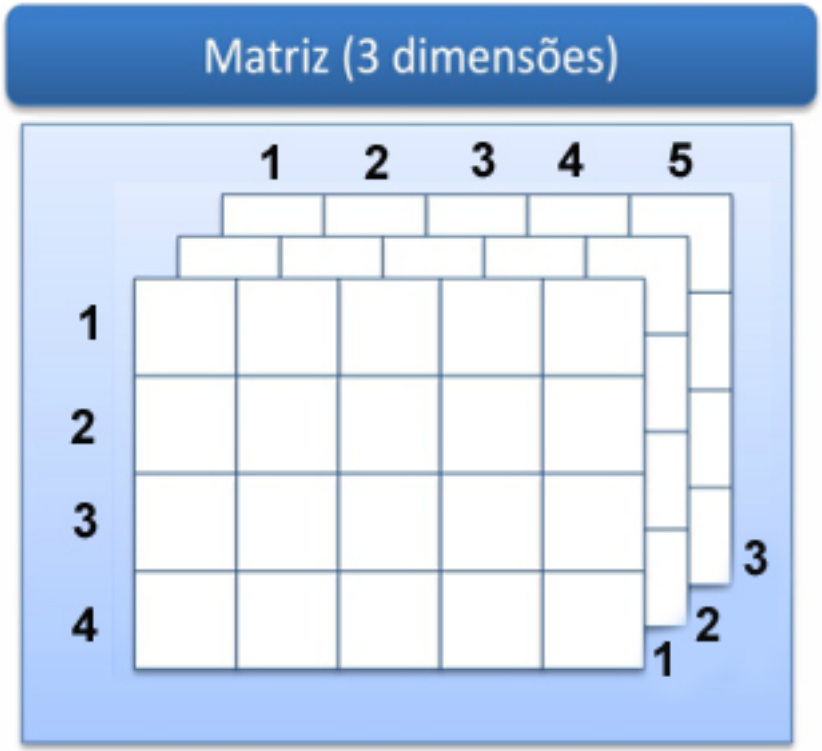
- Uma matriz é um vetor composto por mais de uma dimensão. Mesmo tendo mais de uma dimensão, na matriz, a indexação ocorre da mesma forma que no vetor





Tipos de dados avançados: matriz

- Uma matriz pode ter n dimensões (sempre maior que 1);
- Na prática, normalmente este número não passa de 3



Declaração de matrizes

A declaração de uma matriz é feita de forma semelhante à do vetor.

Exemplo:

Quantidade de colunas (3)

PESSOAS: VETOR [1..2,1..3] **DE CARACTERE** → Tipo de dado

Nome da Matriz

Tamanho da dimensão (2)

A declaração acima corresponde à declaração de uma matriz de 2 linhas e 3 colunas.

Atribuindo valores a uma matriz

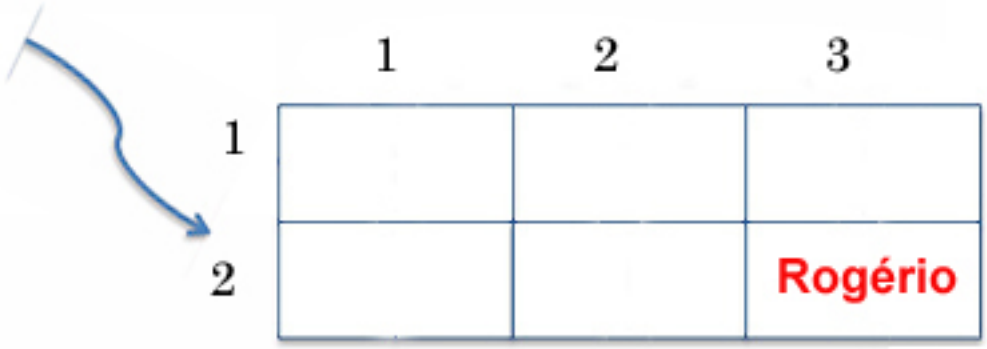
Da mesma forma que no vetor, para atribuir um valor na matriz é necessário fornecer sua posição

- A posição depende do número de dimensões

Exemplo:

PESSOAS: VETOR [1..2,1..3] **DE CARACTERE**

PESSOAS[2,3] <- "Rogério"



Leitura de valores de uma matriz

Para extrair o valor de uma matriz, é preciso identificar a posição onde ele se encontra.

```
algoritmo "matriz pessoas"
// Autor :  Rogério Fernandes
// Data : 05/02/2016

var

PESSOAS: VETOR [1..2,1..3] DE CARACTERE
//matriz pessoas: 2 linhas por 3 colunas

inicio
// Seção de Comandos
PESSOAS[2,3] <- "Rogério"
//a matriz está recebendo o valor "Rogério" na linha 2, coluna 3

escreva (PESSOAS[2,3])
//O comando escreva está sendo usado para acessar a informação na posição

fimalgoritmo
```

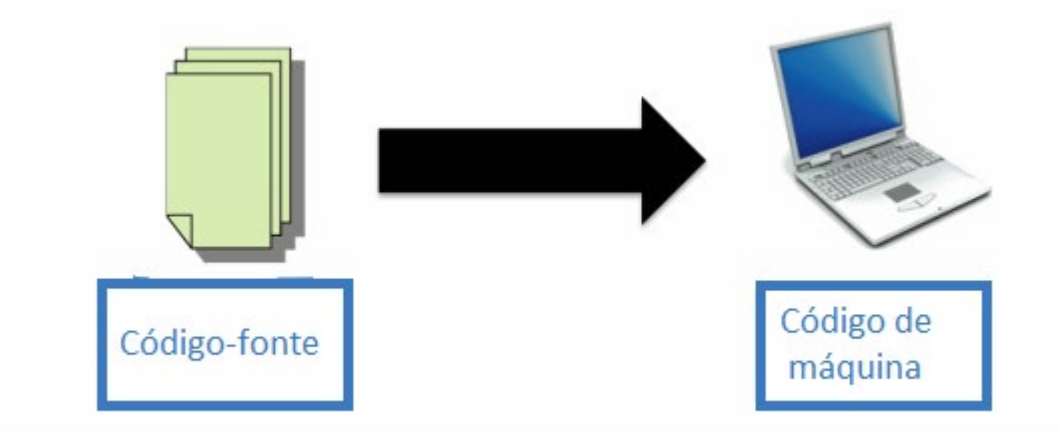
Linguagens de Programação

As linguagens de programação são utilizadas na construção de programas computacionais. A partir do momento que conhecemos a lógica de programação, estamos aptos a dar o próximos passos:

- Aprender alguma linguagem de programação;
- Traduzir os conceitos lógicos para um programa de verdade.

Linguagens de Programação

O funcionamento de um programa está condicionado ao seu código fonte, que é escrito em alguma linguagem de programação.



Linguagens de Programação

Existem inúmeras linguagens de programação:

Java, C, C++, C#, Pascal, Cobol, PHP, JavaScript, CSS, Python, Objective-C...

Descrição da vaga

A Sunset está com oportunidade para Desenvolvedor/ Programador Front End Pleno com foco em criação de banner animado utilizando HTML5/CSS3.

Requisitos

- Programar em HTML5, CSS3, JS e jQuery;
- Sólidos conhecimentos de animação utilizando CSS3;
- Domínio do software WebDesigner do Google;
- Adaptar/recortar layouts do Photoshop para a criação de HTML e CSS;
- Contato anterior com projetos responsivos para desktops, tablets e mobile;
- Desenvolvimento HTML crossbrowser (IE7, IE8, IE10, Chrome, Firefox, Safari, Opera, etc).

Linguagens interpretadas e compiladas

Linguagem interpretada:

O código de máquina é transformado em linguagem de máquina durante a execução do programa (linha a linha).

Exemplos de linguagens interpretadas: PHP e JavaScript

Linguagem compilada:

O código fonte é transformado em linguagem de máquina antes de ser executado.

Exemplos de linguagens compiladas: Java, Pascal, C

No caso específico do Java, o código fonte é compilado para uma linguagem intermediária e depois executado (máquina virtual).

Paradigmas de programação

A forma de programar varia entre as linguagens de programação. Entre os diversos paradigmas de programação, dois são mais comuns:

Estruturado: programas são divididos em procedimentos e funções. Exemplos: C, PHP

Orientado a objetos: programas onde o foco são os elementos denominados objetos, que interagem uns com os outros. Exemplos: Java, PHP

Níveis de abstração das linguagens

Quanto mais próxima da linguagem de máquina, mais baixo é o nível da linguagem, no caso das linguagens de alto nível ocorre exatamente o oposto, elas são mais intuitivas.

Existem três níveis de abstração:

1. **Baixo** (Exemplo: Assembly)
2. **Médio** (Exemplo: C, C++)
3. **Alto** (Exemplo: Java, Pascal)

Outras variações entre linguagens

Cada linguagem de programação tem suas particularidades (nenhuma linguagem de programação é igual a outra).

- É importante compreender o que é possível e o que não é possível fazer com determinada linguagem.
- Embora alguns conceitos seja universais (variáveis, estruturas de decisão, estruturas de repetição, etc.), a sintaxe para estas estruturas vai variar de uma linguagem para outra, implicando na forma de escrever um algoritmo.

Outras variações entre linguagens

Entre os principais aspectos que podem variar de uma linguagem para outra destacam-se:

- Tipos primitivos existentes;
- Declaração de identificadores;
- Valores de variáveis atribuídos automaticamente ou não;
- Variáveis com ou sem tipo de dado definido;
- Operadores;
- Estruturas de controle;
- Sensíveis ou não a letras maiúsculas e minúsculas (case sensitive).

Lista de Exercícios

A seguir, uma lista composta por 14 exercícios é disponibilizada, a resolução dos exercícios tem como objetivo exercitar o raciocínio lógico.

Importante, a avaliação do aproveitamento do curso se dará através da resolução dos exercícios disponibilizados.

Exercício 1

Crie um algoritmo que:

a) Solicite que o usuário digite a distância percorrida;

b) Solicite que o usuário digite a quantidade de combustível gasto;

c) Calcule o consumo de combustível e;

d) Exiba a mensagem: O consumo médio de combustível do seu veículo é: ‘combustível gasto’

Exercício 2

Crie um algoritmo que:

a) Solicite que o usuário digite a nota do 1º Bimestre;

b) Solicite que o usuário digite a nota do 2º Bimestre;

c) Calcule a média e;

d) Exiba o valor da média;

e) Se a média for maior ou igual a 7 exiba a mensagem "Aluno aprovado";

f) Se a média for menor que 4 exiba a mensagem "Aluno reprovado";

g) Se não for nenhum dos dois casos anteriores, o sistema deverá exibir a mensagem "Aluno em recuperação".

Exercício 3

Escreva um algoritmo que:

1. Conte de 2 a 20;
2. Exiba os números em ordem crescente;
3. O incremento no valor deve ser de + 2, ou seja, a sequência é 2 em 2 (2, 4, 6, ..., 20)

Exercício 4

Escreva um algoritmo que:

1. Leia dois valores inteiro distintos;
2. Informe qual dos dois valores é o maior.

Exercício 5

Escreva um algoritmo que:

1. Receba um número;
2. Diga se este número está dentro de um intervalo entre 100 e 200.

Exercício 6

Escreva um algoritmo que:

1. Receba a idade da pessoa;
2. Considere a idade a partir de 18 anos como maior de idade;
3. Informe se a pessoa é maior ou menor de idade, dependendo da idade informada.

Exercício 7

Escreva um algoritmo que:

1. Receba o valor que foi depositado;
2. Exiba o valor com rendimento após um mês;
3. Considere o juro fixo da poupança em 0,70% a. m.

Exercício 8

Escreva um algoritmo que:

1. Receba o número do mês;
2. Mostre o mês correspondente;
3. Se o usuário digitar um número de mês inexistente o sistema deve exibir “Mês inválido!”.

Exercício 9

Escreva um algoritmo que:

1. Leia o **nome** de um vendedor;
2. Receba o valor do seu **salário fixo**;
3. Receba o total de **vendas** efetuadas por ele no mês;
4. Calcule o **salário final** que é composto pelo salário fixo + 10% de comissão sobre as vendas;
5. Exiba em uma mesma mensagem, o salario final e o nome do vendedor

Exercício 10

Escreva um algoritmo que:

1. Receba a quantidade de horas trabalhadas;
2. Calcule e exiba o salário de um professor;
3. Se o Professor for Nível 1 R$25,00 por hora/aula;
4. Se o Professor for Nível 2 R$30,00 por hora/aula;
5. Se o Professor for Nível 3 R$38,00 por hora/aula

Exercício 11

Escreva um algoritmo que:

1. Leia o custo de fábrica de um automóvel;
2. Informe o custo ao consumidor;
3. Aplique 45% de impostos sobre o custo de fábrica;
4. Aplique a margem de lucro de do revendedor (20% sobre o custo de fábrica);
5. Informe o valor final do automóvel.

Exercício 12

Escreva um algoritmo que:

1. Possua um vetor com 4 posições do tipo inteiro;
2. Solicite que um valor seja digitado;
3. Escreva o valor na posição 2 do vetor;
4. Solicite que o segundo valor seja digitado;
5. Escreva o segundo valor na posição 4 do vetor;
6. Realize a soma dos dois valores;
7. Exiba o resultado.

Exercício 13

Escreva um algoritmo que:

1. Possua um vetor com 4 posições do tipo inteiro;
2. Solicite que um valor seja digitado;
3. Escreva o valor na posição 3 do vetor;
4. Verifique se o numero é par ou ímpar;
5. Verifique se o numero é positivo ou negativo
6. Exiba o resultado.

Exercício 14

Escreva um algoritmo que:

1. Possua uma matriz com 3 linhas e 3 colunas do tipo inteiro;
2. Solicite que um valor seja digitado;
3. Escreva o valor na posição 1,3 da matriz;
4. Solicite que o segundo valor seja digitado;
5. Escreva o segundo valor na posição 2,2 da matriz;
6. Realize a soma dos dois valores;
7. Exiba o resultado.

Proposições e Conectivos

- Na área da Lógica o encadeamento de idéias é chamado de ARGUMENTO
- Dado que um ARGUMENTO é formado por uma seqüência de proposições/afirmações, o seu desfecho é chamado de CONCLUSÃO e os passos anteriores são as PREMISSAS
- A Lógica objetiva analisar se a CONCLUSÃO é uma conseqüência lógica das PREMISSAS

Proposições

- A PROPOSIÇÃO é um conjunto de palavras que exprimem um pensamento dentro de certo contexto, podendo ser VERDADEIRO ou FALSO:
  - O IFRN é uma escola de ensino médio
  - O IFRN é uma escola de nível técnico
  - O IFRN é um instituto de ensino superior
  - O IFRN tem apenas o nível médio
  - O IFRN não tem apenas o nível técnico e médio

Proposições

- Não são proposições:
  - Interjeições
    - Nossa, que prova difícil!
    - Ei!
  - Questões
    - Você vai almoçar na cantina hoje?
    - O que?
  - Frases Imperativas
    - Leia o livro que indiquei!
    - Guarde o celular!

Proposições

- Princípios das proposições:
  - IDENTIDADE
    - Uma poposição verdadeira é verdadeira, uma proposição falsa é falsa (0 e 1 no computador)
  - NÃO-CONTRADIÇÃO
    - Uma proposição não pode ser verdadeira e falsa simultaneamente
  - TERCEIRO EXCLUÍDO
    - Uma proposição OU será verdadeira OU será falsa

      (aprenderemos os operadores lógicos em breve)

Proposições

- As proposições podem ser SIMPLES:
  - P = todo aluno é do IFRN
  - Q = todos os alunos são estudiosos
- Ou COMPOSTAS:
  - P = todo aluno é do IFRN e estudioso
  - Q = o ENEM foi difícil e muitos se deram mal

Conectivos

- Os conectivos da lógica são:
  - NOT (NÃO) na programação é !
  - AND (E) na programação é &&
  - OR (OU) na programação é ||
  - XOR (OU exclusivo)
  - => (então)
  - <=> (se e somente se)

Proposições

- Com mais de uma proposição podemos usar os conectivos e formar:
  - Conjunção: P AND Q
  - Disjunção: P OR Q
  - Disjunção Exclusiva: P XOR Q
  - Condicionais: P => Q
  - Bicondicionais: P <=> Q

Tabela-Verdade

- Proposições lógicas compostas podem se tornar complicadas a partir de certo ponto
- Essa ferramenta ajuda a identificarmos a conclusão de uma proposição composta
- Vejamos agora as tabelas-verdade dos conectivos já estudados juntamente com os exemplos no quadro para esclarecer as dúvidas

Tabela-Verdade

- Vejamos agora a tabela do ^ (E):

P	Q	R (P AND Q)
V	V	V
V	F	F
F	V	F
F	F	F

Tabela-Verdade

- Vejamos agora a tabela do v (OU):

P	Q	R (P OR Q)
V	V	V
V	F	V
F	V	V
F	F	F

Tabela-Verdade

- Vejamos agora a tabela do v (OU EXCLUSIVO):

P	Q	R (P XOR Q)
V	V	F
V	F	V
F	V	V
F	F	F

Tabela-Verdade

- Vejamos agora a tabela do $\Rightarrow$ (CONDICIONAL):

P	Q	R (P => Q)
V	V	V
V	F	F
F	V	V
F	F	V

Tabela-Verdade

- Vejamos agora a tabela do ⇔ (BICONDICIONAL):

P	Q	R (P <=> Q)
V	V	V
V	F	F
F	V	F
F	F	V

Tabela-Verdade

- Resumindo:

Estrutura Lógica	Verdadeiro Quando	Falso Quando
P AND Q	Ambos são V	Um dos dois for F
P OR Q	Um dos dois for V	Os dois forem F
P XOR Q	Apenas um for V	Ambos forem F ou V
P => Q	Nos demais casos	P é V e Q é F
P <=> Q	P e Q forem iguais	P e Q forem diferentes

Exercício

- Faça uma tabela verdade para cada conectivo visto explicando suas proposições P e Q em formato textual também
- O formato textual precisa fazer sentido lógico em português
- Desenhe os respectivos conjuntos matemáticos

Ordem de Precedência

- Ao montar as tabelas-verdade precisamos atentar para a precedência dos conectivos:
  - Parênteses
  - Negações
  - Conjunções e Disjunções na seqüência
  - Condicionais
  - Bicondicionais

Negações

- Para negar uma proposição utiliza-se o til
  - Como o leitor de telas não lê o til, utilizarei a palavra ‘não’ no lugar do til
  - Na forma textual basta colocar o ‘não’:
  - Diego é professor
  - Diego NÃO é professor (negativa)
- Na negativa da negativa remove-se o ‘não’:
  - Diego não é professor
  - Diego é professor (negativa)

Negações

- Já a negação de proposições compostas seguem regras próprias, de acordo com o conectivo empregado
- A negação de uma **conjunção**:
  - p AND q
  - não p OR não q (negativa)
- Forma textual:
  - Diego é estudante e professor
  - Diego não é estudante ou não é professor (negativa)

Negações

- Vejamos a tabela verdade:
  - P = diego é estudante
  - Q = diego é professor

P	Q	P AND Q	não (P AND Q)	P OR não Q
V	V	V	F	F
V	F	F	V	V
F	V	F	V	V
F	F	F	V	V

Negações

- A negação de uma **disjunção**:
  - p OR q
  - Não p AND não q (negativa)
- Forma textual:
  - Diego tem conta na Origin ou tem conta ne Steam
  - Diego não tem conta na Origin e não tem conta na Steam (negativa)

Negações

- Vejamos a tabela verdade:
  - P = diego tem conta na Origin
  - Q = diego tem conta na Steam

P	Q	P OR Q	não (P OR Q)	não P OR não Q
V	V	V	F	F
V	F	V	F	F
F	V	V	F	F
F	F	F	V	V

Negações

- A negação de uma **condicional**:
  - p => q
  - p AND não q (negativa)
- Forma textual:
  - Se Diego joga então ele tem conta na Steam
  - Diego joga e não tem conta na Steam (negativa)

Negações

- Vejamos a tabela verdade:
  - P = diego tem conta na Origin
  - Q = diego tem conta na Steam

P	Q	P => Q	não (P => Q)	não P OR não Q
V	V	V	F	F
V	F	F	V	V
F	V	V	F	F
F	F	V	F	F

Negações

- A negação de uma **bicondicional**:
  - p <=> q
  - (p AND não q) OR (q OR não p) (negativa)
- Forma textual:
  - Diego joga se e somente se ele tem conta na Steam
  - Diego joga e não tem conta na Steam OU Diego tem conta na Steam e não joga (negativa)

Negações

- Vejamos a tabela verdade:
  - P = diego joga
  - Q = diego tem conta na Steam

P	Q	P AND não Q	Q AND não P	(P AND não Q) OR (Q AND não P)
V	V	F	F	F
V	F	V	F	V
F	V	F	V	V
F	F	F	F	F

Tautologia

- TAUTOLOGIA é toda proposição composta em cuja última coluna da sua tabela só aparece V (VERDADE)
- Em outras palavras, independentemente do valor de P e Q, R é sempre verdade.
- Exemplo:

P	não P	P OR não P
V	F	V
F	V	V

Contradição

- CONTRADIÇÃO é toda proposição cuja última coluna só tem F (FALSO)
- Em outras palavras, não importa o valor de P e Q, R é sempre FALSO:

P	não P	P AND não P
V	F	F
F	V	F

Contingência

- Chama-se CONTINGÊNCIA toda proposição cuja última coluna tem pelo menos um V e um F
- Ou seja, tudo que não for TAUTOLOGIA nem CONTRADIÇÃO É CONTINGÊNCIA:

P	Q	R (P => Q)
V	V	V
V	F	F
F	V	V
F	F	V

Exercício

- Faça as seguintes negações:
- **1) (CESPE 2014)** Determine a negação da proposição "Lívia é estudiosa e Marcos decora".
  - Lívia é estudiosa ou Marcos decora
  - Lívia não é estudiosa e Marcos decora.
  - Lívia não é estudiosa ou Marcos decora.
  - Lívia não é estudiosa ou Marcos não decora.
  - Marcos não decora e Lívia é estudiosa.

Exercício

- **2) (CESGRANRIO – 2012)** A negação da proposição "Todo professor de matemática usa óculos" é:
  - Nenhum professor de matemática usa óculos.
  - Ninguém que usa óculos é professor de matemática.
  - Todos os professores de Matemática não usam óculos.
  - Existe alguma pessoa que usa óculos e não é professor de matemática.
  - Existe algum professor de matemática que não usa óculos.

Exercício

- **3) (ESAF – 2013)** A negação da proposição "Brasília é a Capital Federal e os Territórios Federais integram a União" é:
  - Brasília não é a Capital Federal e os Territórios Federais não integram a União.
  - Brasília não é a Capital Federal ou os Territórios Federais não integram a União.
  - Brasília não é a Capital Federal ou os Territórios Federais integram a União.
  - Brasília é a Capital Federal ou os Territórios Federais não integram a União.
  - Brasília não é a Capital Federal e os Territórios Federais integram a União.

Exercício

- Monte as tabelas-verdade e informe se são tautologia, contradição ou contingência
- Exemplo: **(q => p) => (p => q)**

P	Q	P => Q	Q => P	(Q => P) => (P => Q)
V	V	V	V	V
V	F	F	V	F
F	V	V	F	V
F	F	V	V	V

Exercício

- As seguintes proposições são tautologia, contradição ou contingência:
  - Não (p AND não p)
  - p OR não (p AND q)
  - p AND q => (p <=> q)
  - (p AND q) AND não (p OR q)
  - p OR q => p AND q
  - Não p AND (p AND não q)
  - p <=> não p
  - **(q => p) => (p => q)**
  - p OR (q AND não q) <=> p

Equivalências Lógicas

- Duas proposições são logicamente equivalentes quando os resultados de suas tabelas-verdade são idênticos
- A equivalência entre uma proposição P e uma Q é representada por P = Q
- Uma proposição pode ser trocada por uma equivalente para resolução de questões

Equivalências Lógicas

- Equivalências básicas:

Forma Normal	Forma Equivalente
P AND P	P
P OR P	P
P AND Q	Q AND P
P OR Q	Q OR P
P <=> Q	Q <=> P
P <=> Q	(P => Q) AND (Q => P)

Equivalências Lógicas

- Equivalências da condicional:
  - P => Q = não Q => não P
    - Se tem internet então eu jogo CSGO
    - Se não jogo CSGO então não tem internet
  - P => Q = não P OR Q
    - Se estudo então passo de ano
    - Não estudo ou passo de ano

Forma Normal	Forma Equivalente
P => Q	não Q => não P
P => Q	não P OR Q

Equivalências Lógicas

- Equivalências com símbolo da negação
  - São as negações que já estudamos

Forma Normal	Forma Negativa
P AND Q	não P OR não Q
P OR Q	não P AND não Q
P => Q	P AND não Q
P <=> Q	(P AND Q) OR (Q AND nãoP)

Equivalências Lógicas

- Equivalências entre ‘todo’ e ‘nenhum’
  - Nenhum A é B = Todo A é não B
    - Nenhum médico é pobre = todo médico é não pobre
  - Todo A é B = Nenhum A é não B
    - Toda arte é bela = nenhuma arte é não bela

Equivalências Lógicas

- Outras equivalências:

Forma Normal	Forma Equivalente
P AND (P OR Q)	P
P OR (P AND Q)	P

Equivalências Lógicas

- Leis Associativas:

Forma Normal	Forma Equivalente
(P AND Q) AND S	P AND (Q AND S)
(P OR Q) OR S	P OR (Q OR S)

Equivalências Lógicas

- Leis Distributivas:

Forma Normal	Forma Equivalente
P AND (Q OR S)	(P AND Q) OR (P AND S)
P OR (Q AND S)	(P OR Q) AND (P OR S)

Exercício

- Resolva as questões abaixo:
- **(ENGENHEIRO DE PETROLEO PETROBRÁS GESGRANRIO 2010) -** Dos slogans abaixo, o que é equivalente a “Se beber, então não dirija” é
  (A) “Se não dirigir, então beba”.
  (B) “Não beba nem dirija”.
  (C) “Não beba ou não dirija”.
  (D) “Se não beber, então dirija”.
  (E) “Beba e não dirija”.

Exercício

- **ANO: 2015 BANCA: FUNDATEC ÓRGÃO: BRDE PROVA: ANÁLISE DE SISTEMAS - ADMINISTRADOR DE BANCO DE DADOS**

-Nego que Mário ou João são engenheiros. Se Mário não é engenheiro então Mário é agrônomo. Se João trabalha na construção civil então João é engenheiro.

- Deduzimos que é verdadeiro:

  (A) Mário é agrônomo e João é engenheiro

  (B) Mário não é agrônomo e João é engenheiro

  (C) Mário é agrônomo e João não trabalha na construção civil

  (D) Mário é agrônomo e João trabalha na construção civil

  (E) Mário é engenheiro e João trabalha na construção civil

Exercício

- **Utilizando as equivalências lógicas, simplifique as seguintes proposições:**

```
A) (p OR q) AND não p

B) p AND (p => q) AND (p => não q)

C) p AND (p OR q) => (p OR q) AND q

D) não (p => q) AND ((não p AND q) OR não (p OR q))

E) não p => (p OR não (p OR não q))
```

Argumento Lógico

- ARGUMENTO é a afirmação de que um grupo de proposições iniciais resulta em uma proposição final, consequência das primeiras
- As proposições iniciais são as PREMISSAS e a final é a CONCLUSÃO
- Exemplo:
  - P1: Todo adolescente joga video game
  - P2: Todo jogo é divertido
  - C: Todo adolescente se diverte

Argumento Lógico

- Outros exemplos:
  - P1: Todos os jogos são caros
  - P2: Mario é um jogo
  - C: Mario é caro

  - P1: Todo músico é inteligente
  - P2: Fulano é músico
  - C: Fulano é inteligente

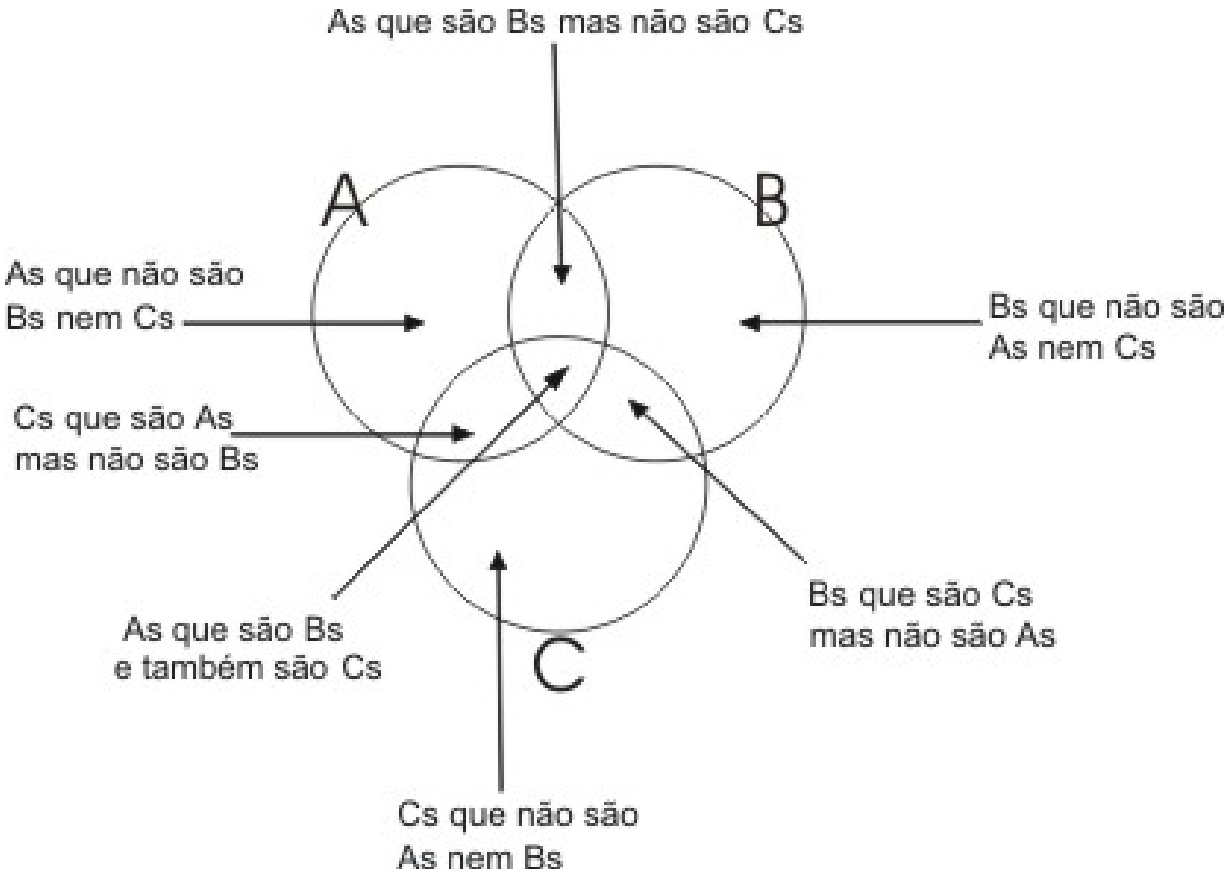
Argumento Lógico

- Argumento Válido:
  - Quando sua conclusão é conseqüência obrigatória de suas premissas
  - Não leva em conta a realidade dos fatos!
- SILOGISMO:
  - P1: todos os homens são peixes
  - P2: nenhum peixe é animal
  - C: nenhum homem é animal

[Sinal de positivo representado por uma mão fechada com o dedo polegar virado para cima]

Argumento Lógico

- SILOGISMO em Diagramas de Venn:



A imagem mostra três conjuntos: A, B e C. Cada um representado por um círculo. Os três conjuntos se conectam entre si, dois a dois: A com B, B com C e C com A. Adicionalmente há uma área central onde os três conjuntos se conectam. Nesta área todos os elementos pertencem aos três conjuntos. É como na intersecção de conjuntos matemáticos.

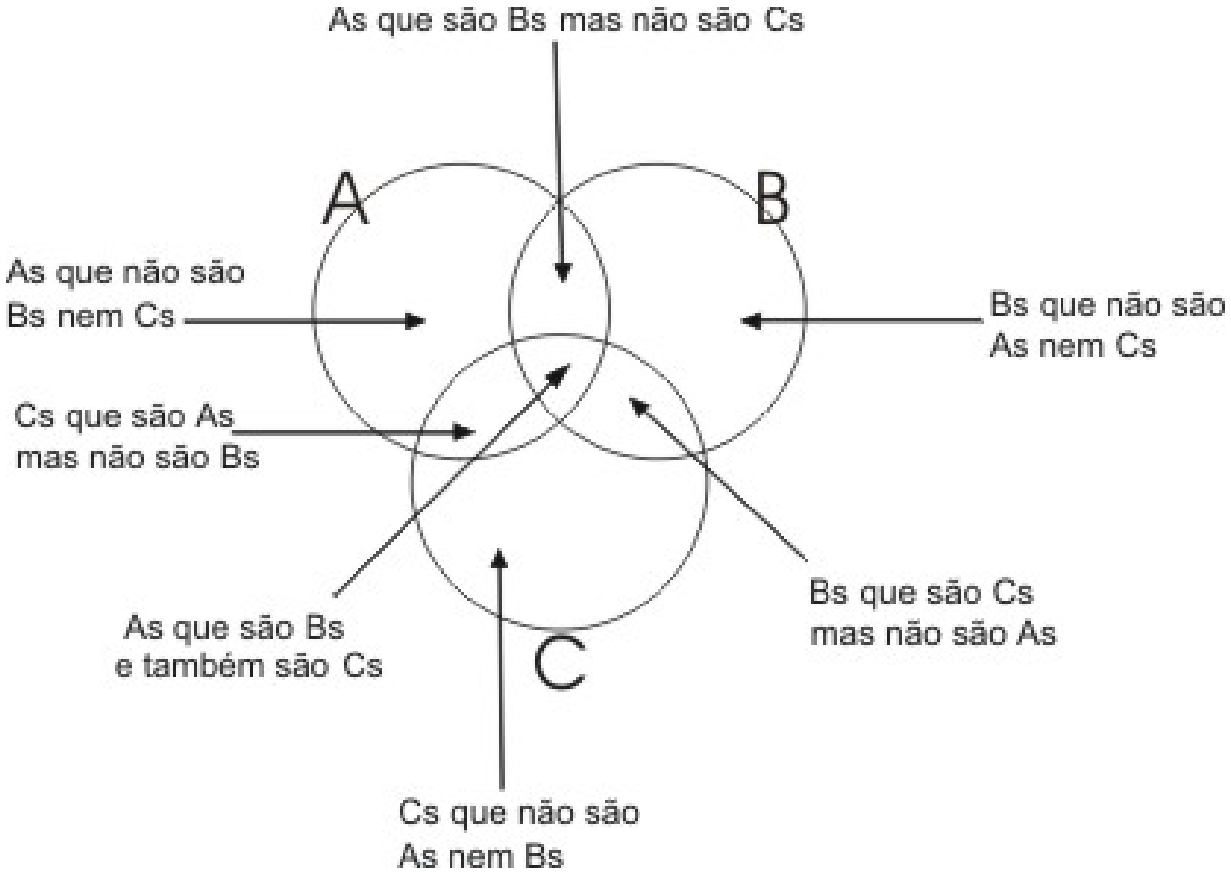
Argumento Lógico

- Argumento Inválido:
  - Quando a verdade das premissas não é suficiente para garantir a verdade da conclusão
  - Novamente a realidade dos fatos não importa
- SOFISMA (ou FALÁCIA):
  - P1: Toda criança gosta de doce
  - P2: Fulano não é criança
  - C: Portanto, Fulano não gosta de doce

[Sinal de negativo representado por uma mão fechada com o polegar virado para baixo]

Argumento Lógico

- SOFISMA em Diagramas de Venn:



A imagem mostra três conjuntos: A, B e C. Cada um representado por um círculo. Os três conjuntos se conectam entre si, dois a dois: A com B, B com C e C com A. Adicionalmente há uma área central onde os três conjuntos se conectam. Nesta área todos os elementos pertencem aos três conjuntos. É como na intersecção de conjuntos matemáticos.
-Mesma imagem do slide 4

Exercício

- Assinale o item que contém um silogismo inválido (sofisma).
  - (A) P1: Algum filósofo é burro
    P2: Todo mundo que sabe física quântica não é burro
    C: Algum filósofo não sabe física quântica
  - (B) P1: Todo cristão não é ateu
    P2: Todo ateu é mau
    C: Todo cristão não é mau
  - (C) P1: Todo lógico é inteligente
    P2: Todo morador de Varginha não é inteligente
    C: Todo lógico não mora em
  - (D) P1: Algum pescador mente
    P2: Todos que gostam de nadar não são pescadores
    C: Algum mentiroso não gosta de nadar
  - (E) Todos os itens representam sofismas.

Exercício

- **Vunesp 2012 - Concurso TJM-SP Analista de Sistemas**

Pergunta: Se afino as cordas, então o instrumento soa bem. Se o instrumento soa bem, então toco muito bem. Ou não toco muito bem ou sonho acordado. Afirmo ser verdadeira a frase: não sonho acordado. Dessa forma, conclui-se que

a) sonho dormindo.

b) o instrumento afinado não soa bem.

c) as cordas não foram afinadas.

d) mesmo afinado o instrumento não soa bem.

e) toco bem acordado e dormindo.

Exercício

- **Cesgranrio/2012 - Concurso Petrobrás – Técnico de Exploração de Petróleo Júnior – Informática**

Pergunta: O turista perdeu o voo ou a agência de viagens se enganou. Se o turista perdeu o voo, então a agência de viagens não se enganou. Se a agência de viagens não se enganou, então o turista não foi para o hotel. Se o turista não foi para o hotel, então o avião atrasou. Se o turista não perdeu o voo, então foi para o hotel. O avião não atrasou. Logo,

a) o turista foi para o hotel e a agência de viagens se enganou.

b) o turista perdeu o voo e a agência de viagens se enganou.

c) o turista perdeu o voo e a agência de viagens não se enganou.

d) o turista não foi para o hotel e não perdeu o voo.

e) o turista não foi para o hotel e perdeu o voo.

Exercício

- Crie 3 sofismas
- Crie 3 silogismos
- Desenhe os Diagramas de Venn para os 6 casos criados
- Escreva as tabelas-verdade para os 6 casos criados

Operadores Lógicos e Relacionais

- Na programação utilizamos operadores para fazer comparações:

Operador	Significado
>	verifica se o número da esquerda é maior que o da direita
<	verifica se o número da esquerda é menor que o da direita
>=	verifica se o número da esquerda é maior OU igual ao da direita
<=	verifica se o número da esquerda é menor OU igual ao da esquerda
&&	retorna verdadeiro se os dois lados forem verdadeiro
||	retorna verdadeiro se pelo menos um dos lados for verdadeiro
==	verifica se o número da esquerda é igual ao da direita
!=	verifica se os números são diferentes

Operadores Lógicos e Relacionais

- Vejamos alguns exemplos:

```
public class Relacional{
    public static void main(String[] args){
        int idadeDeMaior = 18;
        int idadeProfessor = 30;
        int idadeAluno = 15;
        System.out.println(idadeProfessor>idadeDeMaior); //true
        System.out.println(idadeAluno<idadeDeMaior); //true
        System.out.println(idadeProfessor<=idadeAluno); //false
        System.out.println(idadeDeMaior==18 && idadeAluno != 18); //true
        System.out.println(idadeDeMaior>15 || idadeALuno==18); //true
    }
}
```

Operadores Lógicos e Relacionais

- As mensagens impressas podem ser personalizadas:

```
public class Relacional{
    public static void main(String[] args){
        int idadeDeMaior = 18;
        int idadeAluno = 15;
        double salarioMinimo = 980;
        double salarioPessoa = 1200;
        System.out.println("O aluno é de maior?"+ (idadeAluno >= idadeDeMaior));
        System.out.println("O salário da pessoa é o mínimo?"+ (salarioPessoa == salarioMinimo));
    }
}
```

- Também podemos ler os valores do teclado para que o código não fique fixo
- Vejamos um exemplo no próximo slide

Operadores Lógicos e Relacionais

- Exemplo com mensagens personalizadas de acordo com os valores lidos do teclado:

```
public class Relacional{
    public static void main(String[] args){
        Scanner leia = new Scanner(System.in);
        int idadeDeMaior = 18;
        int idadeAluno; //vai ser lido do teclado
        double salarioMinimo = 980;
        double salarioPessoa; //vai ser lido do teclado

        System.out.println("Digite a idade do aluno: ");
        idadeAluno = leia.nextInt();
        System.out.println("Digite o salário da pessoa: ");
        salarioPessoa = leia.nextDouble();

        System.out.println("O aluno é de maior?"+ (idadeAluno >= idadeDeMaior));
        System.out.println("O salário da pessoa é o mínimo?"+ (salarioPessoa == salarioMinimo));
    }
}
```

Exercício

- Leia a idade e imprima se a pessoa é de maior
- Leia o salário e imprima se ganha mais que o mínimo
- Leia a altura e imprima se é maior que 1.8m
- Leia o peso e imprima se é menor que 60kg
- Leia o nome e imprima se é igual ao SEU
- Leia o gênero e imprima se é igual a ‘m’
- **OBS.: Imprima mensagem + true ou false.**

Estruturas Condicionais

- Na programação as estruturas condicionais são utilizadas para verificar se uma condição é verdadeira (como vimos na aula passada) e caso seja **TRUE** o bloco de código entre chaves será executado
- Também é possível adicionar um bloco de código para ser executado caso a condição analisada seja **FALSE**
- Vejamos alguns exemplos no próximo slide

Estruturas Condicionais

- Verificando se uma pessoa é de maior:

```
public class EstruturasCondicionais{
    public static void main(String[] args){
        int idadeDeMaior = 18;
        int idadeAluno = 15;
        if(idadeAluno > idadeDeMaior){
            System.out.println("O aluno é de maior");
        }else{
            System.out.println("O aluno é de menor");
        }
    }
}
```

Estruturas Condicionais

- Verificando se uma pessoa ganha mais que um salário mínimo:

```
public class EstruturasCondicionais{
    public static void main(String[] args){
        double salarioMinimo = 980;
        double salarioPessoa = 1200;
        if(salarioPessoa > salarioMinimo){
            System.out.println("Essa pessoa ganha mais que o mínimo");
        }else{
            System.out.println("Essa pessoa ganha menos que o mínimo");
        }
    }
}
```

- Podemos colocar um IF dentro de outro, são IF encadeados ou aninhados:

Estruturas Condicionais

```
public class EstruturasCondicionais{
    public static void main(String[] args){
        double salarioMinimo = 980;
        double salarioPessoa = 1200;
        int idadePessoa = 20;
        int idadeDeMaior = 18;
        if(salarioPessoa > salarioMinimo){
            System.out.println("Essa pessoa ganha mais que o mínimo");

            if(idadePessoa < idadeDeMenor){
                System.out.println("Ganha mais que o mínimo e é de menor");
            }else{
                System.out.println("Ganha mais que o mínimo e é de maior");
            }

        }else{
            System.out.println("Essa pessoa ganha até um salário mínimo");
        }
    }
}
```

Estruturas Condicionais

- Podemos verificar 2 informações de uma vez utilizando && ou || :

```
public class EstruturasCondicionais{
    public static void main(String[] args){
        double salarioMinimo = 980;
        double salarioPessoa = 1200;
        int idadePessoa = 20;
        int idadeDeMaior = 18;
        if(salarioPessoa > salarioMinimo && idadePessoa >= idadeDeMaior){
            System.out.println("Ganha mais que o mínimo e é de menor");
        }

        if(salarioPessoa < salarioMinimo || idadePessoa < idadeDeMaior){
            System.out.println("Essa pessoa ganha menos que o mínimo ou é de menor");
        }

    }
}
```

Estruturas Condicionais

- Comparando se um nome é igual ao outro:

```
public class EstruturasCondicionais{
    public static void main(String[] args){
        Scanner leia = new Scanner(System.in);
        String nome = “Diego”; //o leitor não lê as aspas, mas String sempre tem.
        String nomeDigitado;

        System.out.println(“Digite o seu nome: ”);
        nomeDigitado = leia.nextLine();

        if(nome.equals(nomeDigitado)){
            System.out.println(“Seu nome é igual ao meu!”);
        }
    }
}
```

Estruturas Condicionais

- Podemos utilizar um contador para verificar quantas vezes entrou dentro dos condicionais
- No exemplo do próximo slide, vai verificar se entrou duas vezes no IF e vai imprimir
  “essa pessoa só quer ser eu!”

Estruturas Condicionais

```
public class EstruturasCondicionais{
    public static void main(String[] args){
        Scanner leia = new Scanner(System.in);
        String nome = “Diego”; //o leitor não lê as aspas, mas String sempre tem.
        String nomeDigitado;
        int idade = 30;
        int idadeDigitada;
        int contador = 0;

        System.out.println(“Digite o seu nome: ”);
        nomeDigitado = leia.nextLine();

        if(nome.equals(nomeDigitado)){
               System.out.println(“Seu nome é igual ao meu!”);
             contador++;
        }

        System.out.println(“Digite sua idade: ”);
        idadeDigitada = leia.nextInt();

        if(idade == idadeDigitada){
               System.out.println(“Sua idade é igual à minha!”);
             contador++;
        }

        if(contador==2){
            System.out.println(“Essa pessoa só quer ser eu!”);
        }
    }
}
```

Exercício

- Leia o nome da pessoa e verifique se é igual ao seu, imprimindo: "**Pessoa com nome igual**" ou "**Pessoa com o nome diferente**"
- Faça a mesma coisa para idade, peso, altura e mais 3 informações sobre a pessoa à sua escolha (série, música, jogos favoristos etc.)
- Caso a pessoa possua 3 ou mais características iguais às suas, imprima: "**Esta pessoa é bem parecida comigo!**"
- Utilize pelo menos 1 IF aninhado

Laços de Repetição

- Laços de repetição são utilizados para executar um bloco de código uma determinada quantidade de vezes, ou até que uma condição seja satisfeita
- No Java temos 4 laços de repetição: FOR, WHILE, DO-WHILE e FOR EACH
- Na disciplina de algoritmos veremos apenas os 3 primeiros laços, começando pelo FOR

FOR

- O FOR, em português PARA, é utilizado para executar um bloco de código uma determinada quantidade de vezes, baseado no valor de uma variável, geralmente ‘ i ’ :

```
public static void main(String[] args){
    for(int i=1; i<10; i++){
        System.out.println(“Valor de i=“+i);
    }
}
```

```
A saída deste código será:
Valor de i=1
Valor de i=2
Valor de i=3
Valor de i=4
Valor de i=5
Valor de i=6
Valor de i=7
Valor de i=8
Valor de i=9
```

FOR

- Observe o código com calma, a variável ‘ i ‘ é criada com valor = 1, depois é avaliada até seu valor ser < 10, ou seja, 9, e a última parte ‘i++’ quer dizer que o ‘ i ‘ vai ser somado de um em um:

```
public static void main(String[] args){
    for(int i=1; i<10; i++){
        System.out.println(“Valor de i=“+i);
    }
}
```

FOR

- O FOR pode ser utilizado também de maneira decrescente:

```
public static void main(String[] args){
    for(int i=10; i>0; i--){
        System.out.println("Valor de i="+i);
    }
}
```

```
A saída deste código será:
Valor de i=10
Valor de i=9
Valor de i=8
Valor de i=7
Valor de i=6
Valor de i=5
Valor de i=4
Valor de i=3
Valor de i=2
Valor de i=1
```

FOR

- O ‘ i ‘ pode ser incrementado em diferentes valores, como por exemplo de 2 em 2:

```
public static void main(String[] args){
    for(int i=1; i<=10; i+=2){
        System.out.println(“Valor de i=“+i);
    }
}
```

```
A saída deste código será:
Valor de i=1
Valor de i=3
Valor de i=5
Valor de i=7
Valor de i=9

*Observe que a contagem é de 2 em 2, não de 1 em 1.
```

FOR

- Dentro do FOR podemos colocar qualquer tipo de código, inclusive blocos IF:

```
public static void main(String[] args){
    for(int i=1; i<=10; i++){
        if(i%2==0){
            System.out.println(i+“ é um número par“);
        }else{
            System.out.println(i+“ é um número ímpar“);
        }
    }
}
```

FOR

- Saída do código do slide anterior:

```
1 é um número ímpar
2 é um número par
3 é um número ímpar
4 é um número par
5 é um número ímpar
6 é um número par
7 é um número ímpar
8 é um número par
9 é um número ímpar
10 é um número par
```

FOR

- Ao executar qualquer laço de repetição, podemos sair dele executando o comando BREAK e podemos pular uma volta com CONTINUE :

```
public static void main(String[] args){
    for(int i=1; i<=10; i++){
        if(i==5){
            continue;
        }

        if(i==8){
            break;
        }
        System.out.println("i="+i);
    }
}
```

FOR

- Saída do código do slide anterior:

```
i=1
i=2
i=3
i=4
i=6
i=7
```

FOR

- Também podemos utilizar um FOR dentro de outro FOR:

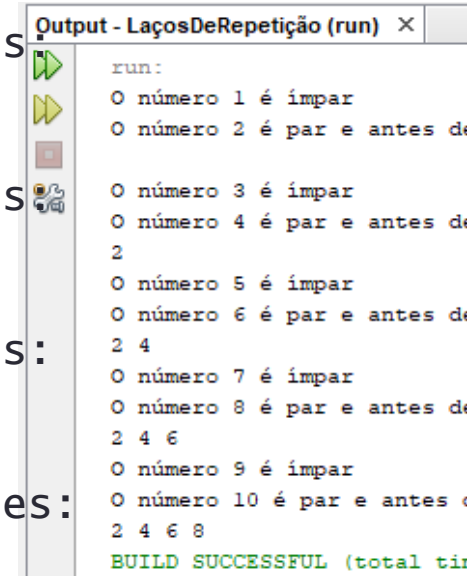
```
public static void main(String[] args){
    for(int i=1; i<=10; i++){
        if(i%2==0){
            System.out.println("O número "+ i + " é par. Pares antes: ");
            for(int j=1; j<i; j++){
                if(j%2==0){
                    System.out.print(j + " ");
                }
            }
            System.out.println();
        }else{
            System.out.println("O número " + i + " é ímpar");
        }
    }
}
```

FOR

- Saída do código do slide anterior:

```
O número 1 é impar
O número 2 é par e antes dele tem os seguintes pares:

O número 3 é ímpar
O número 4 é par e antes dele tem os seguintes pares:
2
O número 5 é ímpar
O número 6 é par e antes dele tem os seguintes pares:
2 4
O número 7 é ímpar
O número 8 é par e antes dele tem os seguintes pares:
2 4 6
O número 9 é ímpar
O número 10 é par e antes dele tem os seguintes pares:
2 4 6 8
```



Exercício

- Escreva um código Java para imprimir de 1 a 100 em ordem crescente, informando se o número é par, ímpar, múltiplo de 3, de 4 ou de 5
- Se o número for múltiplo de 5, escreva outro FOR dentro do IF para imprimir todos os números múltiplos de 5 antes dele em ordem decrescente
- **EXTRA: verifique se os números são PRIMOS!**

WHILE

- O laço de repetição WHILE, em português ENQUANTO, é executado enquanto uma condição for verdadeira:

```
public static void main(String[] args){
    int i = 0;
    while(i<10){
        System.out.println("i="+ i++);
    }
}
```

WHILE

- A saída do código anterior é:

```
i=0
i=1
i=2
i=3
i=4
i=5
i=6
i=7
i=8
i=9
```

*Observe que o 10 não é impresso!

WHILE

- Um erro muito comum quando se utiliza o WHILE é não atualizar a variável que está sendo verificada, o que causa um laço infinito e o programa ‘trava’.
- Por isso o ‘i’ no código anterior está sendo incrementado de um em um DENTRO DO LAÇO!

WHILE

- O WHILE também pode ser utilizado para executar laços dos quais não sabemos o número de repetições necessárias, como por exemplo um MENU:

```
public static void main(String[] args){
    Scanner leia = new Scanner(System.in);
    int op=0;
    while(op!=3){
        System.out.println("1-START GAME");
        System.out.println("2-LOAD GAME");
        System.out.println("3-EXIT GAME");
    }
}
```

WHILE

- Saída do slide anterior para entradas ‘1’, ‘2’ e ‘3’:

```
1-START GAME
2-LOAD GAME
3-EXIT GAME

1

1-START GAME
2-LOAD GAME
3-EXIT GAME

2

1-START GAME
2-LOAD GAME
3-EXIT GAME

3
*Só quando se digita 3 é que sai do programa.
```

WHILE

- Um outro exemplo do uso do WHILE seria para calcular a média de uma turma, da qual não sabemos ainda o número de alunos:

```
public static void main(String[] args){
    Scanner leia = new Scanner(System.in);
    int numeroAlunos=0;
    double mediaTurma=0;
    double mediaAluno=0;
    while(mediaAluno != -1){
        System.out.println("Digite a média do aluno:");
        mediaAluno = leia.nextDouble();
        if(mediaAluno != -1){
            numeroAlunos++;
            mediaTurma += mediaAluno;
         }
    }
    mediaTurma = mediaTurma / numeroAlunos;
    System.out.println("Média da turma = " + mediaTurma);
}
```

WHILE

- Saída do código anterior:

```
Digite a média do aluno:
5
Digite a média do aluno:
5
Digite a média do aluno:
6
Digite a média do aluno:
6
Digite a média do aluno:
-1
Média da turma = 5.5
```

*Observe que foi definido para sair do laço caso a média digitada fosse -1, pois não é uma nota válida

Números Aleatórios

- Em Jogos se utiliza muito a geração aleatória de números para criar objetos em posições aleatórias no cenário ou, mais recentemente, gerar cartas em LOOT
- Em Java podemos utilizar o seguinte código:

```
Random ale = new Random();
int aleatorio = 1+ale.nextInt(100);
```

- Este código gerará um número aleatório entre 1 e 100

Exercício

- Faça um jogo no qual será gerado um número aleatório entre 1 e 10000.
- O usuário poderá chutar até 20 vezes
- Caso ele chute um número menor que o gerado aleatóriamente, imprima “é um número MAIOR”, se ele digitar um maior imprima “é um número MENOR”.
- Quando ele acertar, uma mensagem de PARABÉNS deve ser impressa
- Caso ele não acerte em 20 tentativas imprima “Infelizmente você perdeu!”

DO-WHILE

- O laço de repetição DO-WHILE executa a mesma função do WHILE com uma única diferença: sempre entra no laço

```
public static void main(String[] args){
    int i = 1;
    do{
      System.out.println("i="+ i++);
    } while(i<=10);
}
```

- A condição é avaliada DEPOIS do laço
- Nesse caso vai imprimir de 1 a 10

DO-WHILE

- Vejamos claramente a diferença entre DO-WHILE e WHILE:

```
public static void main(String[] args){
    int i = 1;
    do{
        System.out.println("i="+ i);
    } while(i != 1);
}
```

```
public static void main(String[] args){
    int i = 1;
    while(i!=1){
        System.out.println("i="+ i);
    }
}
```

- O do-while vai imprimir 1 e o while não!

DO-WHILE

- Por isso o DO-WHILE é utilizado quando desejamos entrar pelo menos uma vez no laço de repetição
- Vejamos agora um código para imprimir um tabuleiro de xadrez, onde B são as casas brancas e P as casas pretas

DO-WHILE

- Tabuleiro de Xadrez:

```
public static void main(String[] args){
    int i = 1;
    int j = 1;
    do{
        do{
            if((i+j)%2==0){
                System.out.print("P "); j++;
            }else{
                System.out.print("B "); j++;
            }
        }while(j <=8 );
        System.out.println();
        j=1;
        i++;
    }while(i <=8 );

}
```

DO-WHILE

- Saída do código do slide anterior:

```
P B P B P B P B P B
B P B P B P B P B P
P B P B P B P B P B
B P B P B P B P B P
P B P B P B P B P B
B P B P B P B P B P
P B P B P B P B P B
B P B P B P B P B P
```

Exercício

- Tendo em vista o tabuleiro de xadrez, imprima, utilizando dois DO-WHILE, o tabuleiro e as posições das peças PRETAS:
  T=Torre
  C=Cavalo
  b=Bispo
  r=Rainha
  R=Rei
  p=Peão

P=casa preta, B=branca